

Actividad 4 Taller

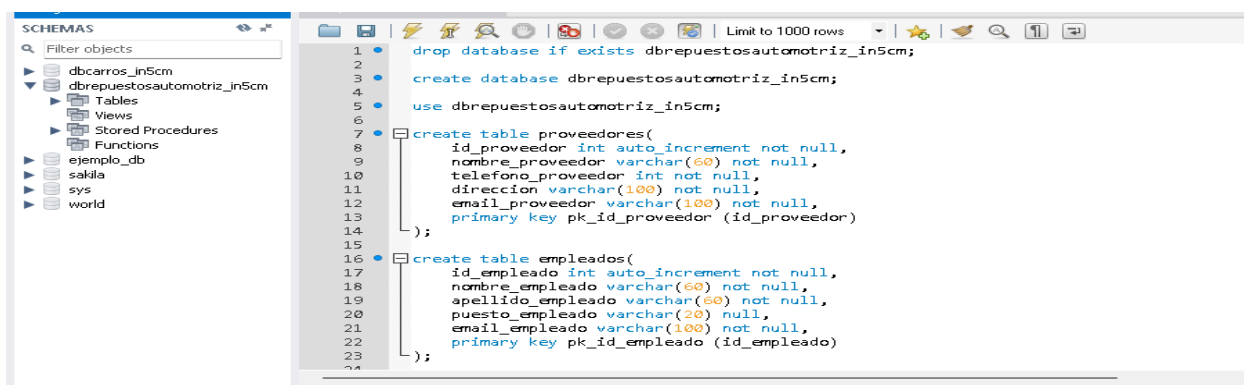
Domingo Romero 2025157

Conexión Spring Boot con MySQL

En esta actividad se realizó la conexión de una aplicación Spring Boot con una base de datos MySQL, se creó la entidad Producto, un repositorio, endpoints REST y se verificó que los datos se guardaran correctamente usando Postman y MySQL.

1. Creación de la base de datos

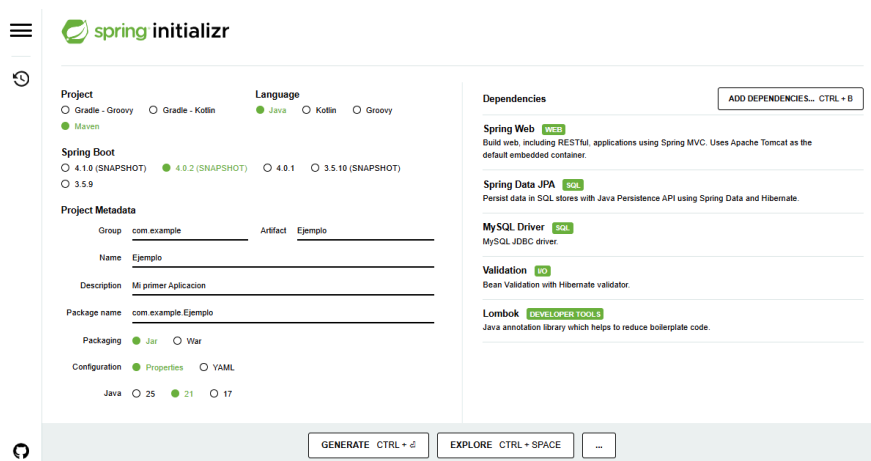
Primero se creó una base de datos en MySQL llamada dbrepuestoautomotriz_in5cm. Esta base se utilizaría para almacenar la información de los productos con los delimitar para cada uno.



```
1 drop database if exists dbrepuestosautomotriz_in5cm;
2
3 create database dbrepuestosautomotriz_in5cm;
4
5 use dbrepuestosautomotriz_in5cm;
6
7 create table proveedores(
8     id_proveedor int auto_increment not null,
9     nombre_proveedor varchar(60) not null,
10    telefono_proveedor int not null,
11    direccion varchar(100) not null,
12    email_proveedor varchar(100) not null,
13    primary key pk_id_proveedor (id_proveedor)
14 );
15
16 create table empleados(
17     id_empleado int auto_increment not null,
18     nombre_empleado varchar(60) not null,
19     apellido_empleado varchar(60) not null,
20     puesto_empleado varchar(20) null,
21     email_empleado varchar(100) not null,
22     primary key pk_id_empleado (id_empleado)
23 );
24
```

2. Configuración de Spring Boot

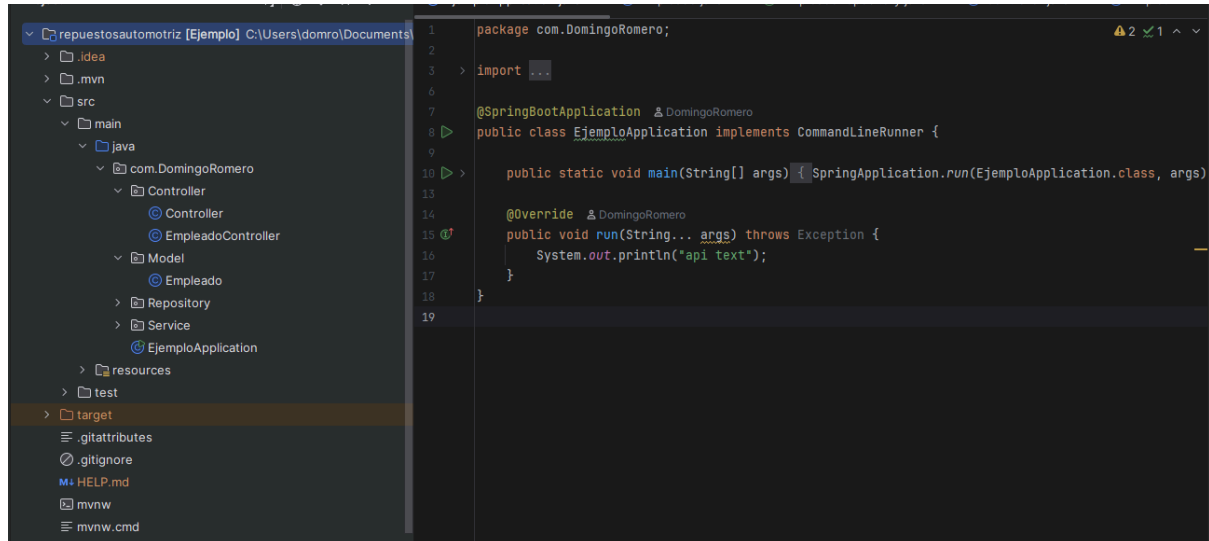
Se configuró el archivo application.properties con la URL de conexión, usuario, contraseña y parámetros necesarios para conectar Spring Boot con MySQL.



The screenshot shows the Spring Initializr web application. The 'Project' section has 'Maven' selected. The 'Language' section has 'Java' selected. The 'Spring Boot' section has '4.0.2 (SNAPSHOT)' selected. The 'Project Metadata' section has 'Group' as 'com.example', 'Artifact' as 'Ejemplo', 'Name' as 'Ejemplo', 'Description' as 'Mi primer Aplicación', and 'Package name' as 'com.example.Ejemplo'. The 'Packaging' section has 'Jar' selected. The 'Configuration' section has 'Properties' selected. The 'Dependencies' section has 'Spring Web', 'Spring Data JPA', 'MySQL Driver', 'Validation', and 'Lombok' selected. The 'GENERATE' button is visible at the bottom.

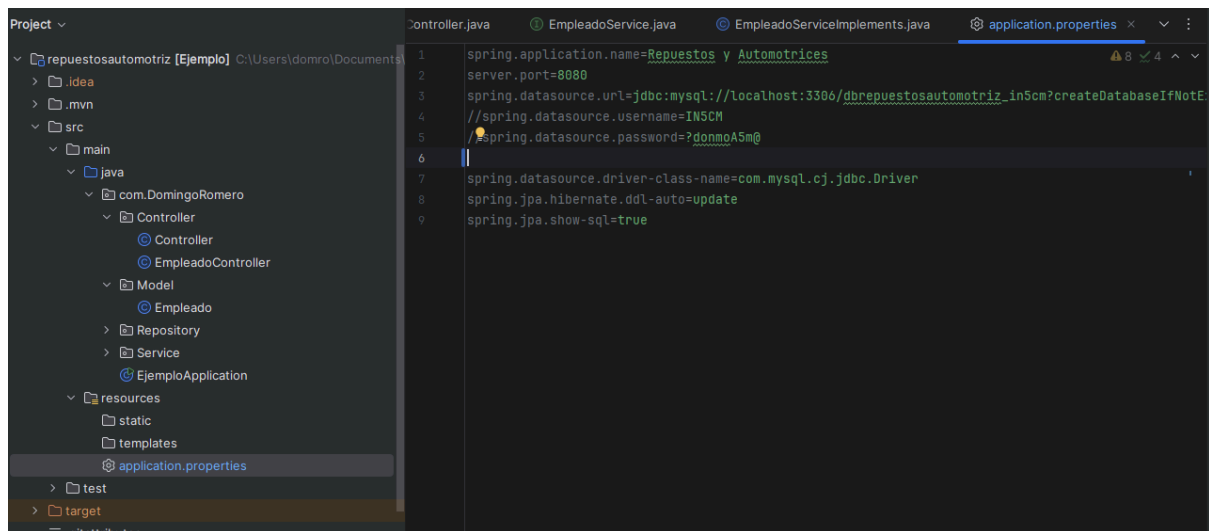
3. Creación de la entidad Producto

Se creó la clase Empleado como entidad JPA. Esta representa la tabla productos en la base de datos y contiene atributos como id, nombre, precio y cantidad.



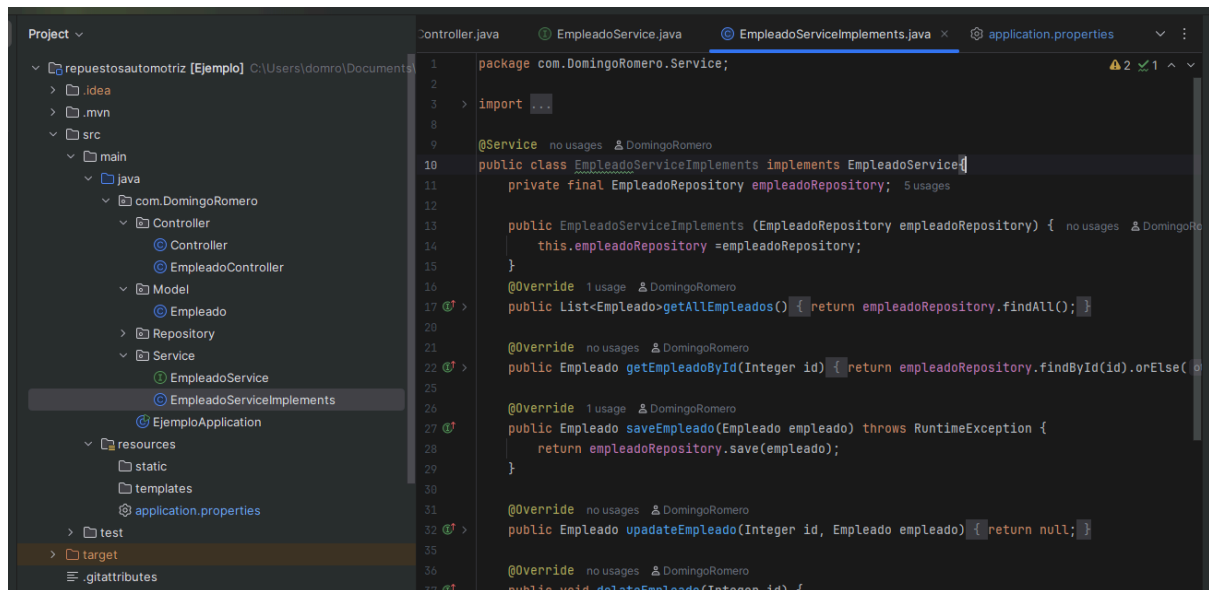
4. Creación del repositorio

Se creó el repositorio EmpleadoRepositor . Este repositorio permite realizar operaciones CRUD sin escribir SQL manualmente.



5. Creación del Service

Se creó la capa Service para manejar la lógica del negocio, permitiendo guardar productos y listar todos los productos registrados.

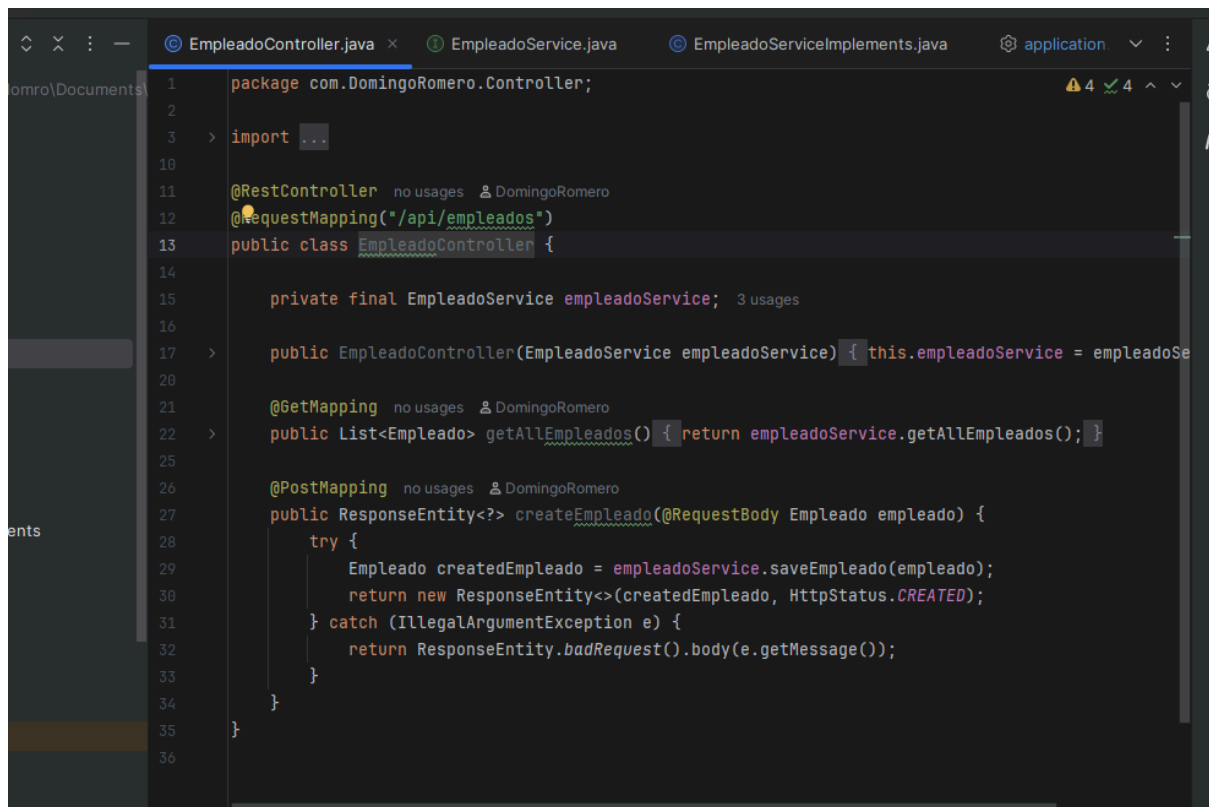


```
Project
└─ repuestosautomotriz [Ejemplo]
   └─ src
      └─ main
         └─ java
            └─ com.DomingoRomero
               └─ Controller
                  └─ EmpleadoController
                     └─ EmpleadoService
                        └─ EmpleadoServiceImpls
```

```
1 package com.DomingoRomero.Service;
2
3 import ...
4
5 @Service
6 public class EmpleadoServiceImpls implements EmpleadoService {
7     private final EmpleadoRepository empleadoRepository;
8
9     public EmpleadoServiceImpls(EmpleadoRepository empleadoRepository) {
10         this.empleadoRepository = empleadoRepository;
11     }
12
13     @Override
14     public List<Empleado> getAllEmpleados() { return empleadoRepository.findAll(); }
15
16     @Override
17     public Empleado getEmpleadoById(Integer id) { return empleadoRepository.findById(id).orElse(null); }
18
19     @Override
20     public Empleado saveEmpleado(Empleado empleado) throws RuntimeException {
21         return empleadoRepository.save(empleado);
22     }
23
24     @Override
25     public Empleado updateEmpleado(Integer id, Empleado empleado) { return null; }
26
27     @Override
28     public void deleteEmpleado(Integer id) { }
```

6. Creación del Controller

Se crearon endpoints REST para registrar productos (POST) y listar productos (GET). Esto permite enviar datos desde Postman o navegador.

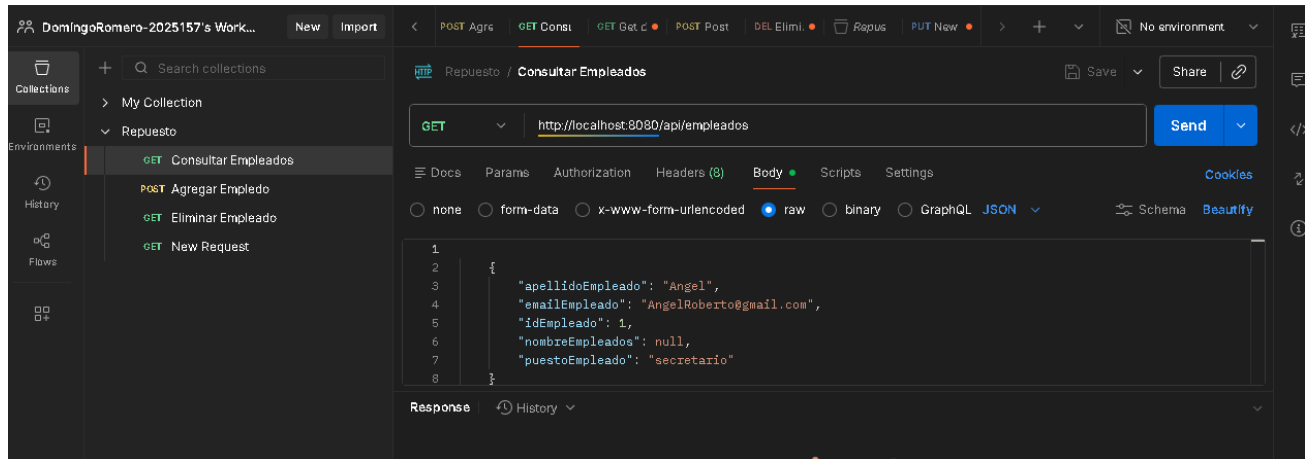


```
EmpleadoController.java
EmpleadoService.java
EmpleadoServiceImpls.java
application.properties
```

```
1 package com.DomingoRomero.Controller;
2
3 import ...
4
5 @RestController
6 @RequestMapping("/api/empleados")
7 public class EmpleadoController {
8     private final EmpleadoService empleadoService;
9
10    public EmpleadoController(EmpleadoService empleadoService) { this.empleadoService = empleadoService; }
11
12    @GetMapping
13    public List<Empleado> getAllEmpleados() { return empleadoService.getAllEmpleados(); }
14
15    @PostMapping
16    public ResponseEntity<?> createEmpleado(@RequestBody Empleado empleado) {
17        try {
18            Empleado createdEmpleado = empleadoService.saveEmpleado(empleado);
19            return new ResponseEntity<>(createdEmpleado, HttpStatus.CREATED);
20        } catch (IllegalArgumentException e) {
21            return ResponseEntity.badRequest().body(e.getMessage());
22        }
23    }
24 }
```

7. Pruebas realizadas

Se utilizó Postman para registrar productos mediante peticiones POST y luego consultar los productos con peticiones GET. También se verificó en MySQL que los datos se almacenaron correctamente.



Conclusión

Se logró conectar Spring Boot con MySQL, crear una entidad, repositorio y endpoints REST. Además, se comprobó que los datos se guardan y se recuperan correctamente.

TRELLO :

<https://trello.com/invite/b/6983d44a828e98936efdd53b/ATTI380033ec8424c8d0b1c9a43ba777d2248FBC30E0/mi-trablero-de-trabajo>

GITHUB

<https://github.com/domingoromero-2025157/repuestosautomotriz.git>